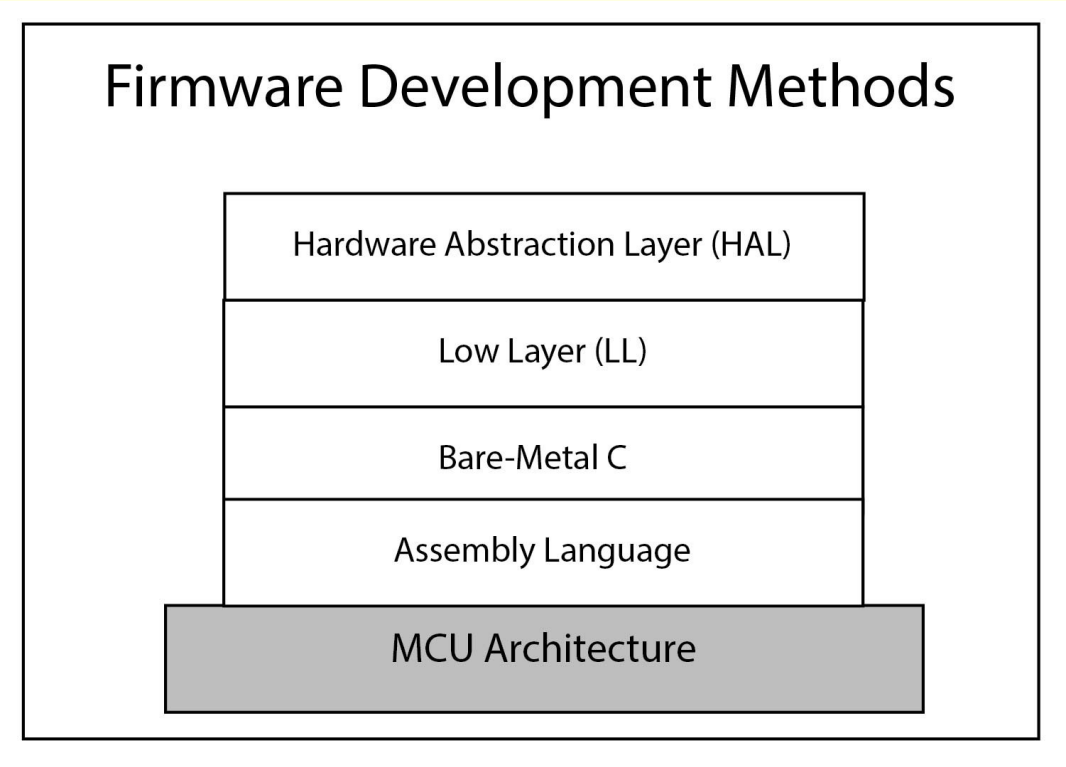
CPE 439 Lecture 02

Review

RTOS Intro/Overview

RTOS Task Intro

Firmware Development



The various levels which you can program the MCU

The Big Overview

- Systems Design: You're using a digital circuit with controlled by software to solve a problem
  - One of many ways to solve a problem
  - CPE 133: hardware control
  - CPE 233: low-level program control
  - CPE 316: higher-level program control
- What the systems did: the waited for input and reacted accordingly (with outputs?)

Operating Systems

- OS: the interface between hardware and the outside world
  - You’ve written these in CPE 133, 233, & 316
- The circuits you designed did stuff
  - Waited for inputs & responded
  - Waited for “events”

- The circuit could be modeled as set of “tasks”
  - Handling input events
  - Handling required outputs

Operating Systems

- Tasks in courses
  - CPE 133: One task? *(easy)*
  - CPE 233: Two tasks? *(easy)*
  - CPE 316: Multiple tasks *(not so easy)*

- More tasks per system mean??? EASY!
  - When does it gets complicated?
    - When response times matter
  - Still not a hard problem except…
    - In a resource constrained environment
      - Particularly using single processor

Operating Systems

- Where we're at:
  - Resource constrained environment
  - Single processor
  - Many tasks
- Solution: add an RTOS to firmware stack
  - RTOS: an operating system designed specifically for in use in real-time systems
  - Real-Time System: a system in which correct operation depends on both logical outcomes and the time at which the result is produced

The RTOS

- Provides a set of software tools to help process events in real time (if possible)
  - Provides opportunity for fast responses
    - Satisfying real-time requirements
  - Gives appearance of multitasking
    - Can more easily manage many tasks

- Making an RTOS-based system:
  - Modeling system as a set of tasks
  - Give each task a priority
  - Ensure the RTOS is always executing the task with the highest priority

Real-Time System definitions

- Real-time system: a system where the response to a given event is deterministic
  - A non-deterministic system: events are responded to with no time constraints
  - Not meeting time constraints means system has failed.
  - A real-time operating system is an OS that provides a deterministic way of executing the code associated with tasks.

Real-Time System Types

- Pure hardware: no code executing
  - Pros:
    - fast timing
    - low power
    - parallelism
  - Cons:
    - not flexible
    - less people can do
    - ASICs are expensive

Real-Time System Types

- <u>Bare-Metal Firmware:</u> (code executing)
  - Pros:
    - Programmer has complete control
    - Flexible
  - Cons:
    - Constrained to relatively simple systems
    - Real-time only in simple system
- Note: complex bare-metal firmware starts resembling RTOS

Real-Time System Types

- <u>RTOS-Based Firmware:</u> (code executing)
  - Pros:
    - Supports real-time concepts (responsive)
    - OS decides the most important task to run
  - Cons:
    - Abstracted to higher level
    - RTOS slow execution
    - RTOS grows program size
    - Can be complicated to make real-time with dire resource constraints.

Types of RTOSes

- Types based on importance of responding to events “in time”
- Hard Real-Time Systems: a system that must needs to meet its deadlines 100% of the time. Missed deadlines: the system failed.
  - Example: medical devices
- Firm Real-Time Systems: a system that needs to make deadlines nearly all of the time.
  - Examples: video or audio where occasional incorrect data is not noticed and/or does not matter
- Soft Real-Time Systems: Systems where it’s permissible to miss some deadlines as the system only promises to do its best to meet deadlines.
  - Typical computer operating system

RTOS: Fast Intro

- Quick and high-level overview
  - Related to what you already know
  - Hopefully somewhat intuitive
- The RTOS approach
  1) Create/define tasks
  2) Start the scheduler
- Task: an abstraction of a program in an RTOS

RTOS: Fast Intro

- Creating Tasks:
  - Define what each task needs to do
  - Each task is viewed as a small program itself

- Start the Scheduler: the part of a RTOS that decides which task will run
  - Part of “kernel”
  - Based on task state and priority
  - Ensures highest priority task will always be the one receiving processing time

Typical Non-RTOS “Super Loop”

```
int main (void)
{
   for (;;)
   {
     // check for input or deferred processing
     // check for input or deferred processing
     // check for input or deferred processing
     // …
   }
}
```

- Endless loop
- Program never ends
- Basically polling

RTOS Task Code

```
void ATaskFunction(parameters)
{
   for (;;)
   {
      // code defining task
   }
   // should not get here
}
```

- Defined as C function
- Endless loop
- Code never exits for loop

RTOS Multiple Task Code

```
void ATaskFunction (parameters)
{
   for (;;) {
      // code defining task
   }
}  // should not get here


void AnotherTaskFunction (parameters)
{
   for (;;) {
      // code defining task
   }
}   // should not get here

int main (void)
{
   while (1)  {
    Create_task(ATaskFunction);
    Create_task(AnotherTaskFunction);

    start_scheduler();
   }
} // should not get here
```

- Many endless loops
- The scheduler decides which task will be run

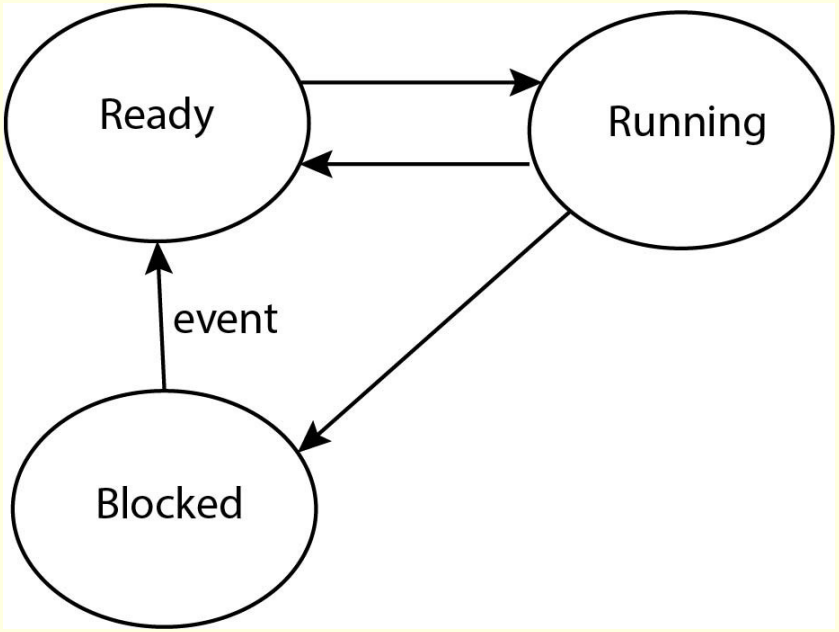
Task States

- Tasks can be in one of three main states

- Running: currently receiving processing time
- Ready: task has something to do
  - But a higher priority task is currently in running state
- Blocked: task has nothing to do (no need for processor time)
  - Typically waiting for some event

Task States

- Running: task receiving processing time
- Ready: task has something to do but waiting for higher priority task to complete
- Blocked: task has nothing to do



Task are designed to "block themselves"

Scheduler responsible for all state changes

Daily Terms and Definitions

- RTOS: an operating system designed specifically for in use in real-time systems
- Real-Time System: a system in which correct operation depends on both logical outcomes and the time at which the result is produced
- Event: a detectable occurrence or change in state that the system monitors, such as user input or hardware interrupts
- Task: an abstraction of a program in an RTOS
- Scheduler: The RTOS part that decides which task will run based on the state and priority of the task
- Kernel: a minimalist operating system. For RTOS, that would include the scheduler, which includes the context switching mechanism.
- Task states: how the kernel classifies the status of each state. The most common states are ***running***, ***ready***, *and* ***blocked***